

Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model

Takuya Sogawa

2026-05-20

Contents

Interface-Led Architecture (ILA)	2
A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model . . .	2
Abstract	2
Keywords	2
1. Introduction	3
2. Background and Related Work	3
2.1 Contract-First Development	3
2.2 Component-Based Software Engineering	3
2.3 AI-Assisted Software Development	3
2.4 AIKernel	4
3. Definition of Interface-Led Architecture (ILA)	4
3.1 Contract, Invariant, and Interface	4
4. Structural Decomposition Model	5
4.1 Overview	5
5. Interface Decomposition, Unit Composition, and Skeleton Formation	6
5.1 Interface Decomposition	6
5.2 Unit Composition	6
5.3 Skeleton Formation	6
6. Use-Case Driven Contract Testing	7
7. AI-Assisted Implementation	7
7.1 AI as a Bounded Implementation Agent	7
7.2 Prompt-Embedded Engineering Heuristics	8
7.3 Controlled Generation Boundary	8
8. Metric-Driven Refinement	8
8.1 The Average 20 Lines / Maximum 100 Lines Rule	9
8.2 Structural Metrics	9
8.3 Refinement Loop	9
9. Human-AI Role Separation	10
9.1 AI Responsibilities	10
9.2 Human Responsibilities	10
9.3 Architectural Ownership	10
10. Validation Using AIKernel	11
10.1 AIKernel Overview	11
10.2 Mapping Between ILA and AIKernel	11
10.3 Pipeline Alignment	11
10.4 Context and ROM Externalization	12
10.5 Contract and Provider Model	12
10.6 ReplayLog and Deterministic Boundaries	12
11. Discussion	13
11.1 Language Independence and Applicability to Python	13
12. Limitations	14
12.1 Increase in the Number of Interfaces	14
12.2 Dependency on Architectural Quality	14
12.3 AI Capability Variance	15

12.4 Lack of Empirical Validation	15
13. Conclusion	15
References	16

Interface-Led Architecture (ILA)

A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model

Version: v1.0.0
Status: Zenodo-ready manuscript
Author ORCID: <https://orcid.org/0009-0009-7499-2595>
DOI: 10.5281/zenodo.20290614
Primary validation target: AIKernel.NET
License: CC BY 4.0

Abstract

With the widespread adoption of Large Language Models (LLMs), software development is transitioning from a process where humans directly write implementations to one where humans design structures while AI assists in implementation, testing, and refinement. Although code generation, test generation, refactoring, and documentation have already been accelerated by AI, incorporating probabilistic AI as an implementation agent makes it difficult to adequately maintain responsibility boundaries, contracts, invariants, and architectural stability using traditional software engineering alone.

In this paper, we propose **Interface-Led Architecture (ILA)** as a software development methodology adapted for the era of AI-assisted development. ILA is a methodology that builds software from interfaces, contracts, and responsibility boundaries rather than from implementation. A system is decomposed into Interfaces as the smallest responsibility boundaries; semantically related Interfaces are composed into Units; and multiple Units are connected to form a Skeleton. Implementations are generated, replaced, and refined within the constraints defined by this Skeleton and its Contracts.

At the core of ILA are Interface Decomposition, Unit Composition, Skeleton Formation, Contract-First Development, Use-Case Driven Contract Testing, AI-Assisted Implementation, Metric-Driven Refinement, and Human-AI Role Separation. AI helps stabilize baseline quality (the lower bound) through implementation generation, test generation, duplication reduction, and metric improvement. Meanwhile, humans retain architectural sovereignty and define upper-bound quality by designing abstractions, boundaries, responsibility divisions, Contracts, Invariants, and Governance.

This paper positions **AIKernel** as the validation environment for ILA. AIKernel is an AI execution and governance substrate equipped with Pipeline, Context, ROM, Contract, Provider, ReplayLog, and deterministic execution boundaries. ROM externalizes human design intent and Contracts as reusable knowledge, while ReplayLog serves as a reproducibility foundation to ensure that AI-driven implementation modifications do not break existing Contracts.

ILA is not restricted to C# / .NET. In Python, responsibility boundaries, Contracts, and Skeletons can be explicitly defined using Protocols, Abstract Base Classes, pydantic models, TypedDicts, validation rules, Pipelines, and DAGs. Therefore, ILA is a structural software development methodology applicable across multiple major development environments in the AI era.

The central thesis of this paper is summarized in the following statement:

In the AI era, implementation becomes probabilistic.
However, architecture must remain deterministic.

Keywords

Interface-Led Architecture; AI-assisted software development; Contract-first development; Software architecture; AIKernel; deterministic execution; ReplayLog; ROM; human-AI collaboration; software governance

1. Introduction

For decades, software engineering has evolved around human comprehension, design judgment, and implementation capability. Object-Oriented Programming, Layered Architecture, Domain-Driven Design, Component-Based Design, and Microservices all assume that humans understand the structure, divide responsibilities, and maintain the implementation.

However, with the advancement of LLMs, this premise is changing. AI can now rapidly generate code, create tests, propose refactoring, explain specifications, and draft documentation. Consequently, much of the implementation work is shifting from being written directly by humans to being generated by AI and evaluated and governed by humans.

At the same time, AI-driven implementation presents inherent challenges. Because AI behaves probabilistically, its generated outputs for the same objective can fluctuate. Furthermore, a lack of context can cause AI to misunderstand responsibility boundaries, apply excessive or insufficient abstraction, and produce redundant implementations or inconsistent structures.

This issue should not be treated merely as a limitation of AI performance. More fundamentally, it stems from the fact that traditional development methodologies do not assume a “probabilistic implementation agent.”

Therefore, software development in the AI era requires asking the following question:

When delegating part of the implementation to probabilistic AI, how should the software structure be designed?

In this paper, we propose **Interface-Led Architecture (ILA)** as a methodology to address this question.

ILA builds software around interfaces and contracts rather than implementation itself. Humans design the system’s skeleton, responsibility boundaries, contracts, and invariants, while AI handles implementation, testing, and refinement within those boundaries.

This structure allows the system’s overall architecture to remain deterministically stable while tolerating the fluctuations of AI-generated implementations.

2. Background and Related Work

2.1 Contract-First Development

Contract-First Development is a design philosophy that defines contracts before implementation. Design by Contract, API-First Development, and Interface-Based Design all share the approach of explicitly specifying expected behaviors and boundaries prior to coding.

ILA inherits these concepts but takes them a step further. In ILA, an interface is not merely a means of abstraction. An interface is a responsibility boundary, a contract boundary, a permission scope for AI implementation, and a target for governance.

In other words, ILA treats contracts not as “design aids” but as the “canonical source of structural truth.”

2.2 Component-Based Software Engineering

Component-Based Software Engineering is a methodology for decomposing a system into reusable units. The idea of separating responsibilities and assembling components is shared with ILA.

However, traditional component-oriented design often assumes that humans will stably maintain the implementation. The scenario where the internal implementation of a component is generated, replaced, or modified by AI was not typically a central assumption.

ILA assumes local implementation generation and replacement by AI. Therefore, implementation units must be small, responsibility boundaries must be clear, and contracts must be fixed.

2.3 AI-Assisted Software Development

AI-assisted development has seen rapid practical application in recent years. LLMs can generate code from specifications, explain existing code, write tests, and suggest refactoring.

However, current AI-assisted development is often Prompt-Centric. That is, the quality of AI output strongly depends on the prompt, the provided context, and the immediate conversation history.

In this state, short-term productivity may improve, but long-term architectural stability is not guaranteed. Code generated by AI might work in the moment, but it can obscure responsibility boundaries and degrade future maintainability.

ILA addresses this problem by fixing the structure first, rather than relying on prompts. AI does not determine the structure; it implements within the constraints defined by the structure.

2.4 AIKernel

AIKernel is an execution and governance foundation for AI applications. AIKernel treats LLMs not merely as API calls but as execution targets governed through Context, Contracts, Execution Boundaries, Providers, and ReplayLogs.

The characteristics of AIKernel are as follows:

- Responsibility flow via Pipeline
- Externalization of Context
- Externalization of design knowledge via ROM
- Enforcement of invariants via Contract
- Separation of implementation units via Provider
- Execution reproducibility through ReplayLog
- Deterministic execution boundaries

These correspond closely to the structural properties required by ILA.

3. Definition of Interface-Led Architecture (ILA)

We define Interface-Led Architecture (ILA) as follows:

ILA is a software development methodology that decomposes a system using interfaces as the smallest responsibility boundaries, fixes invariants through contracts, and confines AI-assisted implementation within a deterministic structure.

In ILA, an interface is not just a type definition. An interface carries multiple meanings:

- Responsibility boundary
- Contract boundary
- Implementation replacement boundary
- AI generation boundary
- Testing boundary
- Governance boundary

For this reason, in ILA, interfaces are designed before implementation. Implementations are generated within the scope defined by the Interface and Contract.

The fundamental philosophy of ILA can be summarized in the following statement:

Implementation may change.

However, contracts and boundaries must remain stable.

While AI can rapidly generate implementations, it cannot reliably take responsibility for validating abstractions or designing boundaries. Therefore, in ILA, humans design the boundaries, and AI handles the implementation within them.

3.1 Contract, Invariant, and Interface

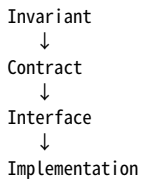
In ILA, Contract, Invariant, and Interface are clearly distinguished.

Contract is the promise of externally observable behavior. For example, what input a Provider receives, what result it returns, and what state it transitions to upon failure belong to the Contract.

Invariant is a condition that must not be violated during execution or state transition. For example, not treating a null response as a success, transitioning to Fail-Closed upon failure, and retaining necessary information in the ReplayLog are treated as Invariants.

Interface is the syntactic entry point to express a Contract as an implementation boundary. Therefore, an Interface is not the Contract itself, but a structure mapping the Contract to an implementable and verifiable boundary.

Through this distinction, the following relationship holds in ILA:

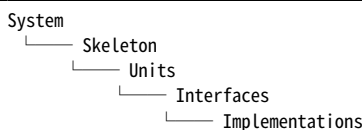


AI can generate the Implementation. However, it must not arbitrarily change the Invariant, Contract, or Interface.

4. Structural Decomposition Model

4.1 Overview

ILA views a system as a hierarchical structure:

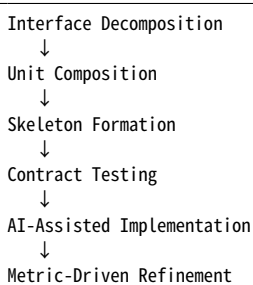


The meanings are as follows:

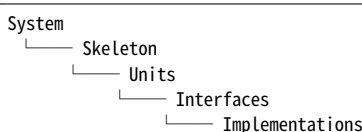
- **Interface:** The smallest responsibility boundary and syntactic expression of a Contract.
- **Unit:** A compositional unit bundling multiple semantically related Interfaces.
- **Skeleton:** The skeletal structure connecting multiple Units and defining the overall execution flow, dependency direction, and governance boundaries of the system.
- **Implementation:** A replaceable implementation that satisfies the Interface and Contract.

What is important here is to distinguish between the development sequence and the completed structural hierarchy.

The development sequence in ILA is expressed as follows:



The completed structural hierarchy, by contrast, can be understood as follows:



In short, ILA starts design from lower-level responsibility boundaries and builds up to higher-level structures. When understanding the completed system, however, the structure is read hierarchically from the Skeleton down to Units, Interfaces, and Implementations.

5. Interface Decomposition, Unit Composition, and Skeleton Formation

5.1 Interface Decomposition

In ILA, an Interface is the smallest responsibility boundary. It is the fundamental unit for implementation generation, test generation, replacement, and validation by AI.

The characteristics of an Interface are as follows:

- Has a single responsibility
- Has a small public surface
- Can be tested independently
- Its implementation can be replaced
- Easy to delegate implementation to AI
- Functions as a syntactic expression of a Contract

Example:

```
public interface IContextProvider
{
    ContextSnapshot Build();
}
```

In this example, `IContextProvider` has only the responsibility of building a Context. Validation, storage, transformation, and caching of the Context should be separated into different Interfaces.

By defining many such small responsibility boundaries, ILA creates a structure where AI can easily implement locally, and humans can govern effectively.

5.2 Unit Composition

A Unit is a compositional unit that bundles multiple semantically related Interfaces.

Example:

```
Context Unit
├── IContextProvider
├── IContextValidator
├── IContextSerializer
└── IContextCache
```

A Unit represents a single domain of responsibility within the system. While each Interface has an independent responsibility, the Unit provides semantic cohesion.

The purpose of a Unit is not merely to organize files. It is to clarify the proximity of responsibilities, stabilize dependencies, and create units that make it easier for AI to generate implementations locally.

The Unit sits between the Interface and the Skeleton, acting as the central compositional unit in ILA.

5.3 Skeleton Formation

The Skeleton is the overarching structural frame of the system. It is not just a collection of Interfaces or Units; it defines the execution flow, dependency direction, and governance boundaries.

Example:

```
[VFS Unit]
  ↓
[ROM Unit]
  ↓
[Context Unit]
  ↓
[Prompt Generation Unit]
  ↓
[Provider Unit]
  ↓
[Execution Result Unit]
```

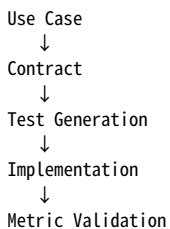
The Skeleton is not a target for AI to change freely. It is the canonical structure of the system and the domain that humans must design and maintain.

AI handles the implementation of the Interfaces positioned inside the Skeleton. Thus, AI in ILA is not the creator of the structure, but an implementer operating within it.

6. Use-Case Driven Contract Testing

In ILA, Contracts are derived from use cases, and tests are generated based on those Contracts.

The basic flow is as follows:



The key here is not to write tests based on the implementation, but to create tests based on the use case and the Contract.

Example:

Use Case:
The Provider returns a deterministically verifiable ExecutionResult.

Contract:

- Must always include an ExecutionStatus
- Must not treat a Null response as a success
- Must retain necessary information in the ReplayLog
- Must transition to Fail-Closed upon failure

By fixing the Contract first in this way, AI can safely generate the implementation. Even if the generated implementations vary, they are replaceable as long as they satisfy the Contract.

Testing in ILA is not merely quality assurance. Tests are executable representations of Contracts and serve as boundaries for governing AI implementations.

7. AI-Assisted Implementation

7.1 AI as a Bounded Implementation Agent

In ILA, AI is a bounded implementation agent. AI is not an entity that freely modifies the architecture; it generates implementations within the scope defined by Interfaces and Contracts.

The roles AI should assume are:

- Implementation generation
- Test generation
- Duplication reduction
- Simplifying redundant code
- Metric improvement
- Reducing boilerplate

On the other hand, the roles AI should not assume are also clear:

- Redefining responsibility boundaries
- Altering the Skeleton
- Arbitrarily changing Contracts
- Changing governance structures
- Relaxing invariants

This separation allows humans to retain architectural sovereignty while leveraging AI's generative capabilities.

7.2 Prompt-Embedded Engineering Heuristics

In ILA, the empirical rules of senior engineers are embedded as Prompt Templates.

Examples:

- One File Per Class / Interface
- Fail-Closed by Default
- Dependency Inversion
- Minimal Public Surface
- Deterministic Naming
- Contract Preservation
- Avoid Hidden Responsibilities

These are not merely coding standards. They are compressed knowledge representations intended to convey organizational or project-specific design decisions to the AI.

AI cannot reliably infer unstated design intent. Therefore, in ILA, the judgment criteria of senior engineers are codified as reusable templates.

In AIKernel, these Prompt-Embedded Engineering Heuristics are connected with ROM (Relation-Oriented Markdown). ROM is a format for externalizing design intent, responsibility boundaries, Contracts, and the rationale behind related decisions as reusable knowledge, rather than as ephemeral prompts.

As a result, a senior engineer's heuristics can be treated as deterministic knowledge injected into AI, rather than ad-hoc instructions. Prompt Templates in ILA become structured design knowledge backed by ROM in AIKernel.

Through this structure, AI does not simply “write code”; it generates implementation based on explicit design intent, responsibility boundaries, and Contracts. Consequently, ROM acts as a medium for transmitting human design judgment to the AI implementation in ILA.

7.3 Controlled Generation Boundary

The boundary between human and AI in ILA can be organized as follows:

[Human] Defines: - Interfaces - Contracts - Units - Skeleton - Responsibility Boundaries - Governance Rules - ROM-based Design Knowledge [AI] Generates: - Implementations - Tests - Refactorings - Metric Improvements

This boundary prevents fluctuations in AI-generated implementations from rippling through the overall system structure.

8. Metric-Driven Refinement

In ILA, metrics are used to continuously evaluate the state of the implementation. The goal here is not simply to make the code look clean.

Metrics are used for:

- Detecting responsibility bloat
- Detecting insufficient decomposition
- Maintaining implementation units that are easy for AI to handle
- Maintaining structures that are easy for humans to understand
- Protecting Contract boundaries

8.1 The Average 20 Lines / Maximum 100 Lines Rule

ILA adopts the following guidelines for implementation units:

- Average around 20 lines
- Maximum under 100 lines

This rule is not a mechanical aesthetic. It is a heuristic to maintain a granularity where responsibilities are easily recognizable for both AI and humans.

When an implementation becomes overly long, the following problems are often lurking:

- Mixing of multiple responsibilities
- Unextracted Interfaces
- Insufficient Unit composition
- Hidden branching
- Ambiguity in Contracts

Therefore, a lengthy implementation is not simply shortened; it is treated as a trigger to review responsibility boundaries.

8.2 Structural Metrics

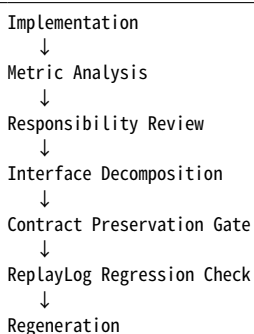
The primary metrics emphasized in ILA are:

Metric	Purpose
Cyclomatic Complexity	Detects bloat in branching structures
Nesting Depth	Detects increases in cognitive load
Coupling	Detects excessive dependency density
Interface Surface Size	Detects bloat in public surfaces
File / Method Size	Detects insufficient responsibility separation

These metrics do not represent absolute correctness in isolation. However, in an environment involving continuous AI implementation generation, they serve as vital signals for early detection of structural degradation.

8.3 Refinement Loop

Refinement in ILA is expressed as the following cycle:



The critical point is that Contracts must not be broken for the sake of improving metrics. In ILA, improvement means making the implementation and responsibility boundaries clearer while maintaining the Contract.

The **Contract Preservation Gate** is the boundary that verifies whether AI modifications have broken existing Contracts or Invariants. On AIKernel, this validation can be performed as a regression check using past execution contexts, inputs, Provider selections, and execution results saved in the ReplayLog.

In this sense, the ReplayLog is not a mere execution history. In ILA, the ReplayLog serves as a regression validation foundation to ensure that AI-driven improvements have not eroded existing contracts.

Metric-Driven Refinement is a mechanism to keep implementations small, simple, and understandable. By contrast, the ReplayLog confirms that these improvements remain consistent with past specifications. By combining the two, ILA achieves both continuous AI-driven improvement and Contract Preservation.

9. Human-AI Role Separation

At the core of ILA is the separation of responsibilities between humans and AI.

AI is a powerful implementation assistant, but it is not the sovereign of the architecture. Humans do not need to write all the code, but they must define what is invariant.

9.1 AI Responsibilities

The area AI should handle is primarily stabilizing the lower bound of quality.

AI excels at:

- Test generation
- Implementation generation
- Deduplication
- Code formatting
- Metric improvement
- Local refactoring

These are effective for stabilizing the baseline quality of the software.

However, AI cannot always correctly grasp the project's overall objectives, responsibility boundaries, and future directions for extension. Delegating high-level design decisions to AI can therefore become a cause of structural degradation.

9.2 Human Responsibilities

The area humans should handle is designing the upper bound of quality.

Humans define:

- Abstractions
- Boundary design
- Responsibility division
- Interfaces
- Units
- Contracts
- Skeleton
- Governance
- Invariants
- Design knowledge externalized via ROM

What is particularly important is not defining “what can be changed,” but rather “what must not be changed.”

The designer in the AI era is not an artisan who implements everything by hand, but an architect who defines an implementable structure.

9.3 Architectural Ownership

In ILA, Architecture Ownership remains with humans.

AI expands implementation capacity. However, it is the human who defines the canonical source of architectural truth.

This separation mitigates the following issues:

- Architecture Collapse
- Prompt-Dependent Drift

- Abstraction Entropy
- Responsibility Leakage
- Contract Erosion

ILA assumes collaboration between human and AI, but this collaboration is not an undifferentiated blending. Humans define the structure, and AI proceeds with implementation inside that structure.

10. Validation Using AIKernel

10.1 AIKernel Overview

ILA is not a methodology limited to AIKernel. ILA represents general principles for structural design in AI-assisted development.

AIKernel, on the other hand, is a concrete execution foundation that makes these principles verifiable at run-time through context management, contract enforcement, Provider separation, and ReplayLogs.

Therefore, the positioning of AIKernel in this paper is not as the sole implementation of ILA, but as a validation environment demonstrating its effectiveness.

AIKernel is an execution and governance foundation for AI applications. It treats LLMs not merely as external APIs, but as execution targets controlled via Context, Contract, Provider, ReplayLog, and Governance.

The primary components of AIKernel are:

- Pipeline
- Context
- ROM
- Contract
- Provider
- ReplayLog
- Deterministic Execution Boundary

These correspond directly to the structures of ILA.

10.2 Mapping Between ILA and AIKernel

The relationship between ILA and AIKernel can be mapped as follows:

ILA Concept	AIKernel Mapping
Skeleton	Execution Pipeline
Unit	Context / VFS / Provider / Governance areas
Interface	Provider interfaces / pipeline contracts
Implementation	Concrete providers / pipeline steps
Contract	Invariants and validation rules
ROM	Externalized design knowledge
ReplayLog	Reproducibility and regression boundary

Through this mapping, AIKernel functions as an environment to validate the abstract theories of ILA at the implementation level.

10.3 Pipeline Alignment

The execution structure of AIKernel can be conceptually represented as follows:



Each stage has a clear responsibility. This aligns exactly with the concepts of Interface / Unit / Skeleton in ILA. The Pipeline fixes the flow of responsibility. Context externalizes implicit context. ROM canonicalizes knowledge about design intent and Contracts. Contract fixes invariants. Provider offers the unit for implementation replacement. ReplayLog supports execution reproducibility.

10.4 Context and ROM Externalization

In AIKernel, Context is not confined to implicit conversation histories or ephemeral Prompts. Context is externalized in forms such as ROMs, Snapshots, DTOs, and ReplayLogs.

ROM, in particular, corresponds to the externalization of design knowledge in ILA. By maintaining responsibility boundaries, design intent, Contracts, and related rationales as ROMs, the context passed to the AI becomes a verifiable knowledge foundation rather than a transient natural language instruction.

This yields the following benefits:

- Reduction of Hidden State
- Verifiability of execution context
- Reusability of design intent
- Clarification of Contracts
- Deterministic Replay
- Control via Governance
- Traceability of AI outputs

In ILA, the context provided to AI must be explicit. AIKernel satisfies this requirement as an execution foundation.

10.5 Contract and Provider Model

The Provider in AIKernel can be understood as the relationship between Interface and Implementation in ILA.

Example:

```
IModelProvider
├── OpenAIProvider
├── LocalProvider
└── Future Providers
```

`IModelProvider` is the Interface, and the respective Providers behind it are replaceable Implementations.

Providers are replaceable as long as they satisfy the common Contract. This structure ensures that AIKernel does not depend on a specific model or vendor.

From the perspective of ILA, a Provider must satisfy the following:

- Responsibilities are defined by the Interface
- Behaviors are fixed by the Contract
- Implementation is replaceable
- It does not break the Skeleton of the entire Pipeline

This property is one of the reasons AIKernel is highly suitable as a practical environment for ILA.

10.6 ReplayLog and Deterministic Boundaries

The ReplayLog in AIKernel is a crucial structure to confine probabilistic AI execution into a verifiable form.

ReplayLog retains the following:

- Input context
- Execution conditions

- Provider selection
- Execution results
- Failure states
- Information necessary for reproduction

This allows the execution process to be saved in a verifiable form, even if the AI's output is probabilistic.

From the perspective of ILA, the `ReplayLog` is the chronological audit trail of a `Contract`. That is, it records “at that point in time, under which `Contract`, based on which `Context`, which `Provider` returned which result.”

The `ReplayLog` is also deeply related to Metric-Driven Refinement. When AI shortens, decomposes, and reduces the complexity of an implementation, those changes must preserve the `Contract`. Because `AIKernel` can save past execution conditions and results via `ReplayLog`, it can verify whether the improved implementation has not broken existing specifications.

Therefore, the `ReplayLog` in `AIKernel` provides a deterministic safety boundary for the Refinement Loop in ILA. This is essential for maintaining specification reproducibility and `Contract` continuity while permitting AI-driven implementation improvements.

This is a vital governance mechanism for software in the AI era.

11. Discussion

ILA shifts the center of software development from implementation to structure.

In traditional development, implementation ability was at the heart of quality. Excellent developers were those who wrote excellent code, understood complex implementations, and modified them as necessary.

However, in the AI era, the cost of generating implementation itself is rapidly dropping. At the same time, the importance of design judgment—what to implement, where to place responsibilities, and what to make invariant—is increasing.

ILA is a methodology that responds to this shift.

In ILA, the value of software development transitions to the following:

- From volume of implementation to division of responsibility
- From code generation to `Contract` design
- From Prompt engineering to structural design
- From local optimization to Skeleton stability
- From personalized judgments to reusable design disciplines
- From implicit knowledge to ROM-based design knowledge
- From execution results to reproducibility via `ReplayLog`

In this sense, ILA does not reject AI-driven development. Rather, to use AI safely, it clarifies the areas that humans must design.

`AIKernel` realizes the philosophy of ILA as an execution foundation. It externalizes human design judgment via ROM, re-validates modifications to AI-generated implementations via `ReplayLog`, and fixes invariants via `Contract`. Consequently, AI is treated not as a free generative agent, but as an implementation agent governed deterministically.

11.1 Language Independence and Applicability to Python

ILA is not a methodology unique to C#. While this paper uses C#'s `interface` as an example to explain the connection with `AIKernel.NET`, the essence of ILA lies not in the syntax of a specific language, but in making responsibility boundaries explicit, fixing them via `Contracts`, and confining AI implementation within those boundaries.

ILA relies not on language features themselves, but on universal design principles: abstraction, boundaries, contracts, and determinism.

ILA can also be applied in Python. Responsibility boundaries equivalent to C#'s `interface` can be expressed in Python using `typing.Protocol`, Abstract Base Classes (ABCs), or explicit function signatures. DTOs and input/output `Contracts` can be represented by `dataclasses`, `pydantic` models, `TypedDict`, and validation rules.

By explicitly defining `Protocols` and `Contracts` in Python, developers can recover structural guarantees comparable in purpose to those provided by C#, even in a dynamic language. What matters here is not the strength

of the type system itself, but that responsibility boundaries and input/output contracts are explicit and maintained through testing and validation.

Additionally, a Unit can be organized as a Python package, module, service boundary, or feature directory. A Skeleton can be expressed as a Pipeline, DAG, workflow graph, or dependency graph. For example, in AI agent development, processes such as retrieval, planning, tool execution, validation, and memory updates can be separated into Units, which are then connected as a Pipeline Skeleton.

The mappings of ILA concepts in Python can be summarized as follows:

ILA Concept	Python Mapping
Interface	Protocol / ABC / function signature
Contract	pydantic model / TypedDict / validation rule
Implementation	concrete class / function / callable
Unit	package / module / service boundary
Skeleton	Pipeline / DAG / workflow graph
Metric	complexity / type coverage / test coverage
Replay	test fixture / execution trace / regression log

As this mapping illustrates, ILA does not depend on C#'s `interface` syntax. In fact, the significance of making responsibility boundaries explicit and Contracts is especially important in dynamic languages like Python.

Because Python is highly flexible, type boundaries, input/output specifications, and responsibility separation can easily become ambiguous during AI-driven implementation generation. Therefore, by adopting ILA, Python code generated by AI can be confined within structures such as Protocols, pydantic models, tests, and DAGs.

Thus, ILA is not a design methodology restricted to .NET / C#, but a structural software development methodology applicable across multiple languages in the AI era.

12. Limitations

12.1 Increase in the Number of Interfaces

Because ILA emphasizes fine-grained interface decomposition, the number of interfaces tends to increase.

This can cause the following issues:

- Increased complexity in repository structure
- Increase in the number of files
- Navigational overhead
- Higher cognitive cost for beginners

Therefore, naming conventions, directory structures, documentation, and searchability are critical when practicing ILA.

12.2 Dependency on Architectural Quality

ILA is predicated on the ability to decompose the system well.

If incorrect responsibility division or excessive abstraction occurs, ILA may amplify complexity instead.

Particular caution is required regarding:

- Proliferation of meaningless Interfaces
- Excessive abstractions without meaningful implementations
- Blurring of Unit boundaries
- Bloating of the Skeleton
- Superficial Contracts
- Obsolescence of design knowledge recorded in ROM

ILA is not a methodology for simply increasing interfaces. It is a methodology for drawing responsibility boundaries correctly.

12.3 AI Capability Variance

The quality of AI-generated implementation depends on the model used, prompts, context, and existing code quality.

High-performance models can generate implementations and tests aligned with Contracts with high accuracy. However, low-performance models or inadequate context may lead to misunderstood Contracts or violated responsibility boundaries.

Therefore, ILA does not trust AI outputs unconditionally. It is built on the premise that outputs must be verifiable through Contracts, Tests, Metrics, ROMs, and ReplayLogs.

12.4 Lack of Empirical Validation

While this paper presented AIKernel as a validation environment, broader empirical evidence is needed to establish the effectiveness of ILA more fully.

Future validation targets include:

- Large-scale enterprise systems
- Distributed systems
- Non-.NET environments
- Implementation comparisons using multiple AI Providers
- Correlation analysis between metric improvement and maintainability
- Effectiveness evaluation of externalizing design knowledge via ROM
- Regression validation of AI-modified implementations using ReplayLog

13. Conclusion

This paper proposed Interface-Led Architecture (ILA) as a software development methodology for the era of AI-assisted development.

ILA decomposes a system using interfaces as the smallest responsibility boundaries, composes semantically related interfaces into Units, and connects multiple Units as a Skeleton. It then fixes invariants via Contracts, confining AI-driven implementation within a deterministic structure.

The core components of ILA are:

- Interface Decomposition
- Unit Composition
- Skeleton Formation
- Contract-First Development
- Use-Case Driven Contract Testing
- AI-Assisted Implementation
- Metric-Driven Refinement
- Human-AI Role Separation

This paper also presented AIKernel as a practical validation environment for ILA. Equipped with Pipeline, Context, ROM, Contract, Provider, ReplayLog, and deterministic execution boundaries, AIKernel structurally aligns with the principles of ILA.

In particular, ROM externalizes human design judgment, responsibility boundaries, and Contracts as deterministic knowledge that can be transmitted to AI systems. The ReplayLog serves as a reproducibility foundation to verify that implementation modifications and improvements by AI have not broken existing Contracts.

Furthermore, ILA is not restricted to C# / .NET. In Python, responsibility boundaries, Contracts, and Skeletons can be explicitly defined using Protocols, pydantic models, validation rules, Pipelines, and DAGs. Therefore, ILA is a software development methodology applicable across major development environments in the AI era.

The conclusion of this paper is as follows:

In the AI era, implementation becomes probabilistic. However, architecture must remain deterministic.

ILA is a structural software development methodology designed to put this principle into practice.

References

1. Meyer, Bertrand. Object-Oriented Software Construction. 2nd ed. Prentice Hall, 1997.
2. Parnas, David L. "On the Criteria to Be Used in Decomposing Systems into Modules." *Communications of the ACM*, 15(12), 1053–1058, 1972.
3. Dijkstra, Edsger W. A Discipline of Programming. Prentice Hall, 1976.
4. Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003.
5. Martin, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
6. Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
7. Brooks, Frederick P. Jr. The Mythical Man-Month: Essays on Software Engineering. Anniversary ed. Addison-Wesley, 1995.
8. Cwalina, Krzysztof, and Brad Abrams. Framework Design Guidelines: Conventions, Idioms, and Patterns for Reusable .NET Libraries. Addison-Wesley, 2005.
9. OpenAI. "GPT-4 Technical Report." *arXiv:2303.08774*, 2023.
10. Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. "Evaluating Large Language Models Trained on Code." *arXiv:2107.03374*, 2021.
11. Amershi, Saleema, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, et al. "Guidelines for Human-AI Interaction." *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019.
12. AIKernel Project Documentation. AIKernel.NET Architecture and Governance Documentation. AIKernel.NET Project, 2026.